



MULUNGUSHI UNIVERSITY
School of Engineering and Technology
Department of Computer Science and IT

Android student registration and lab group management

ICT361 Mobile Application Development | Competency based group lab

Your practical assignment

Develop an Android app that collects student_name, student_number, program_of_study and Lab_group. Students must register, view their group and edit their permitted details. Lecturers must add, view, edit, delete, search and filter student records. Save data locally on the phone and remotely in MySQL through a Node.js server.

Group work requirements

- Complete this lab in project groups of 15 members. Submit one working application and one project package per group. Separately, the application must enforce a maximum of 15 active students in each stored lab group.
- Divide the project group into five teams of three: UI/UX, Android architecture, local storage/sync, backend/database, and testing/integration. Rotate responsibilities at checkpoints. Every member must contribute code, test a feature and explain the complete data flow.
- Use a shared Git repository and a contribution log showing each member's name, student number, task, commit and testing evidence. All members take part in the demonstration and individual questioning.

Practical competencies to develop

By completing this lab, you should be able to:

1. Design accessible forms and navigation, and preserve user work when Android recreates a screen.
2. Implement role-based CRUD, search and filters using Java, a REST API and relational databases.
3. Prevent duplicate students and overfilled groups, including simultaneous requests.
4. Save offline changes, synchronise safely, resolve conflicting edits and demonstrate correctness through tests.

Required stack: Android Studio, Java/XML Views, ViewModel/LiveData, Room, Retrofit/OkHttp, WorkManager, Node.js/Express and MySQL/InnoDB. Use fictitious data and record dependency versions. The lecturer will announce checkpoint and submission dates.

1 Core features and application rules

Student functions

- Register and sign in securely. View your own profile, confirmed group and group occupancy. Save a registration draft when offline; complete account verification online.
- Edit your own name and programme. Request a group change. Request lecturer correction of your student number so that identity and ownership remain protected.

Lecturer functions

- Sign in with a lecturer account and add, view, edit or delete student records. Confirm before deleting, and allow cancellation.
- View students by lab group or programme. Search by name or student number. Combine search, programme and group filters; include an Unassigned option.
- Assign or transfer students to groups, correct student numbers and see group totals. Lecturer actions must also obey the 15-member limit.

Data validation and integrity

- Student name: required, 2–100 trimmed characters; allow ordinary name punctuation and accented letters. Different students may share a name.
- Student number: a required string of nine digits for this lab. Trim outer whitespace, preserve leading zeroes and reject embedded spaces. Enforce uniqueness in MySQL and check locally for early feedback.
- Programme: choose CS, IT or DS from stored reference data. Lab group: choose G01–G04, or show Unassigned. Groups may mix programmes; each has at most 15 active students in total.
- Use an immutable student_id independent of the student number. One active student may belong to only one group. A failed transfer must retain the previous group.
- Create the profile before requesting a place. If a group is full, keep the profile unassigned or retain the old group. An offline request is Pending until confirmed by the server.
- Use soft deletion on the server: hide the record, release its place once, disable its account and retain a deletion marker for sync. Keep its number reserved to prevent accidental re-registration.

Authentication and permissions

Seed one lecturer account and fictitious student claim codes. A student must verify ownership when registering. If the lecturer already entered that student, link the account to the existing profile. Never create a second profile. Hash server passwords; never save passwords on the phone.

The server must check identity, role and record ownership on every protected request. Students must not access another student's record or the full roster. Hiding buttons alone is insufficient. Protect session tokens and clear or lock account data on logout.

2 Practical activities and slide application

Complete these activities in sequence. At each checkpoint, demonstrate the result, record a defect and its correction, and commit the group's work.

Activity A Design the user experience

Sketch registration, sign-in, student profile/group, lecturer roster, student editor and sync/error screens. Apply Unit 2 slides 15–20 and 30–35: predictable Back navigation, labelled fields, useful errors, large touch targets, TalkBack support and flexible layouts. Use Material components and string resources. Test at 200% text size and preserve typed input after errors.

Competency evidence: annotated wireframes and feedback from three classmates completing tasks without coaching. Show at least two improvements based on their observations.

Activity B Build the Android structure

Separate the screen, ViewModel, repository, Room database and API client. Apply Unit 3 slides 6–11, 19–23 and 30–36. Screens display state; repositories coordinate data. Run database and network work off the main thread. Use Room for durable records and drafts; ViewModel alone does not survive process death.

Competency evidence: an architecture diagram, a test with a fake repository, and a demonstration that form/search state survives rotation and saved work survives restart. Relate this to Unit 1 slides 18, 23–25 and 30–31.

Activity C Build the server and database

Draw an ER diagram for students, accounts, programmes and groups. Add stable IDs, unique student numbers, foreign keys, record versions and deletion metadata. Build Node.js/Express routes for registration/login, student CRUD, group assignment, search/filter and sync. Use parameterised SQL and server-side field validation.

Competency evidence: database scripts and an API contract listing method, route, permitted role, request body and responses. Apply Unit 4 slides 15–20 and 30–35, plus Unit 6 slides 21–22 and 29–38. Android communicates with the API, never directly with MySQL. Use HTTPS for the remote backend.

Activity D Integrate and test the mobile experience

Connect the UI to real local and remote data. Paginate lecturer lists; query the backend online and clearly label limited cached results offline. Cancel or ignore old search responses. Handle loading, empty, validation, offline and expired-session states.

Apply Unit 7 slides 41–42 and 44–48 by sharing a lecturer group summary through the Android Sharesheet. Preview only the group label and counts; exclude personal data. No camera, location or Bluetooth permission is needed. Demonstrate safe cancellation and accessible controls.

3 Offline storage and challenging problem solving

Activity E Save and synchronise

Apply Unit 5 slides 29–42. Store proposed edits and their pending operations together in a Room transaction. Keep each operation's unique ID, student ID, account ID, type, payload and base version. Use WorkManager for persistent sync with connectivity constraints and retry backoff; also provide a manual Sync action.

- Show Saved locally, Pending, Syncing, Synced or Action required. Keep confirmed values separate from pending changes. A locally saved group preference does not reserve a place.
- Preserve operation order. Explain how a lecturer's offline create, edit and delete of the same student are replayed or safely combined.
- Use the same operation ID when retrying a mutation. Store the server result durably with the change so a lost response cannot cause a duplicate effect. Reject reuse of that ID with different content.
- Compare base versions before accepting edits. On conflict, preserve the local proposal and show the current server record for review. Do not silently overwrite another person's accepted work.
- Pull authorised updates and deletion markers. A stale offline edit must never recreate a deleted student. Scope local data and queues by account; pause sync on session expiry and prevent replay under another user.

Challenge 1 Two phones request the last place

Start G01 with 14 members. Send two requests at the same time from different clients. Exactly one may succeed; the other must receive GROUP_FULL. Use a MySQL transaction with appropriate locking or an equivalent safe approach. Every assignment, transfer and deletion path must preserve the same rule.

Explain why checking the count only in Android, or counting and inserting in separate unprotected steps, can fail. Repeat the test 20 times from a reset fixture and show final database counts.

Challenge 2 The response disappears

Let the server save a record, interrupt the response, then retry. Restart the server before another retry. Demonstrate that only one record or membership change exists. Explain how your operation receipt makes repeated requests safe.

Challenge 3 Offline changes become outdated

Cache one student on two devices. Edit on one device and save an offline change on the other. Reconnect and demonstrate conflict handling. Repeat with a group that becomes full while offline, and with a lecturer deleting the student remotely.

Competency evidence

Submit the Room model, queue inspection, server responses and database assertions. Show that local saved work survives restart and that two devices converge after accepted sync. Test one Room migration without deleting existing or pending data. Explain the rule enforced by each test.

4 Testing assessment and submission

Minimum demonstration checklist

Test	Expected result
Valid registration and lecturer CRUD	Correct records on phone and server; edits and deletion reflected after sync.
Duplicate number and invalid fields	Duplicate/invalid data rejected; useful feedback; form input retained.
Full group and simultaneous requests	No group exceeds 15; only one client takes the final place.
Failed transfer and repeated deletion	Old group retained on failure; deletion releases only one place.
Offline save and interrupted sync	Saved work survives restart; retry applies each operation once.
Conflicting edit and remote deletion	Conflict shown; accepted edits are not silently lost; no resurrection.
Roles and account switching	Student cannot access others' records; no data or queue leaks.
Search filters and accessibility	Combined filters work; cache scope is clear; usable large text and TalkBack.

For each test, record starting data, steps, expected/actual result and evidence. Use two devices or emulators. Demonstrate local storage and actual MySQL records, not screenshots alone.

Assessment out of 100

Component	Marks
Core student and lecturer functions	20
UI/UX and application of slide principles	15
Architecture and lifecycle handling	10
API security and database integrity	15
Offline storage, sync and conflict handling	15
Testing, documentation and group integration	10
Individual contribution, explanation and live modification	15
Total	100

The shared product earns 85 marks. Each member earns 15 individually: contribution (5), explanation (5) and a supervised code change (5). Failed checks for duplicates, group capacity, permissions or durable local saving must be corrected and demonstrated again.

Responsible use of AI and final hand-in

Keep a short AI/source log: tool or source, question, code affected, changes made and verification. Every member must explain their contribution and complete a short live change without AI or peer help. Never upload real student data or credentials to AI tools.

Submit one package per 15-member project group: Android and Node.js source, APK, database scripts, test evidence, architecture/wireframes, setup README, contribution log and AI/source log. Include a 5–8 minute group demo. Exclude secrets and node_modules. Each member adds a brief reflection on one concept learned and one bug solved.